DATA ENGINEERING

Complete Learning Syllabus

From Foundations to Production-Grade Systems

30+ Technologies • 8 Structured Phases • 8–12 Months

Md. Imrul Hasan
Data Architect

2025 Edition

Table of Contents

Table of Contents 2
1.1 Computer Science Fundamentals 6
Data Structures 6
Algorithms 6
Operating Systems Concepts 6
1.2 Linux & Command Line Mastery 6
Linux Fundamentals 6
Shell Scripting (Bash) 6
Essential CLI Tools 6
1.3 Networking Fundamentals 7
1.4 Version Control – Git 7
1.5 Python Deep Dive 7
Core Python 7
Intermediate Python 7
Data Engineering Python 7
1.6 Scala Fundamentals 8
1.7 Java Basics (for JVM Ecosystem) 8
2.1 RDBMS Concepts & Theory 9
2.2 Advanced SQL 9
Core SQL 9
Window Functions 9
Advanced Constructs 9
2.3 Query Performance & Optimization 9
2.4 PostgreSQL (Primary RDBMS) 10
2.5 MySQL 10
3.1 Hadoop Architecture & History 11
3.2 HDFS (Hadoop Distributed File System) 11
Architecture 11
Operations 11
File Formats & Compression 11
3.3 MapReduce 11
3.4 YARN (Yet Another Resource Negotiator) 12
3.5 Hive 12
3.6 Pig 12
3.7 HBase 12

3.8 Sqoop 12
3.9 Flume 13
3.10 Oozie 13
3.11 Hadoop Security 13
3.12 ZooKeeper 13
4.1 Apache Spark – Core Architecture 14
RDD (Resilient Distributed Datasets) 14
Execution Model 14
4.2 Spark Performance Tuning 14
4.3 PySpark 14
DataFrame API 14
UDFs & Pandas Integration 14
Data I/O 15
4.4 Spark SQL & Catalyst 15
4.5 Spark Structured Streaming 15
4.6 Spark MLlib (Overview) 15
4.7 Apache Flink 15
4.8 Apache Beam 16
4.9 Presto / Trino 16
5.1 Apache Kafka – Core 17
Architecture 17
Producers 17
Consumers 17
Delivery & Storage 17
5.2 Kafka Ecosystem 17
Kafka Operations 17
5.3 Debezium – Change Data Capture (CDC) 17
5.4 Apache NiFi 18
5.5 Cloud Streaming Services 18
AWS Kinesis 18
GCP Pub/Sub 18
Apache Pulsar 18
5.6 Event-Driven Architecture Patterns 18
6.1 Cron & Systemd Timers 20
6.2 Apache Airflow (Deep Dive) 20
Architecture 20
DAG Development 20
TaskFlow API 20

Advanced Features....20
Operations....20
6.3 Luigi....21
6.4 Prefect....21
6.5 Dagster....21
6.6 Mage.ai....21
7.1 MongoDB....22
7.2 Apache Cassandra....22
7.3 Redis....22
7.4 Neo4j....22
7.5 Elasticsearch....22
7.6 Data Warehousing Concepts....23
Architecture Paradigms....23
Dimensional Modeling....23
Slowly Changing Dimensions (SCD)....23
Dimension & Fact Types....23
7.7 Data Modeling Methodologies....23
7.8 Google BigQuery....23
7.9 Snowflake....24
7.10 Amazon Redshift....24
7.11 ClickHouse & Druid....24
ClickHouse....24
Apache Druid....24
7.12 Open Table Formats (Data Lakehouse)....24
8.1 Google Cloud Platform (GCP)....26
8.2 Amazon Web Services (AWS)....26
8.3 Microsoft Azure....26
8.4 Infrastructure as Code (IaC)....27
8.5 Docker & Kubernetes....27
Docker....27
Kubernetes....27
8.6 Data Quality Tools....27
8.7 Data Governance & Cataloging....27
Governance Pillars....27
Data Cataloging....27
Dataplex (GCP)....27
Privacy & Compliance....27
8.8 dbt (Data Build Tool)....28

Core Concepts 28
Advanced 28
8.9 CI/CD for Data Pipelines 28
8.10 Data Architecture Patterns 28
8.11 MLOps Fundamentals (for Data Engineers) 28
8.12 Capstone Projects 29
Additional Resources & Learning Tips 30
File Formats Quick Reference 30
Compression Codecs 30
Learning Path Tips 30
Key Job Requirement Technologies (Quick Map) 30

PHASE 1

Foundations & Prerequisites *(8–12 weeks)*

1.1 Computer Science Fundamentals

Data Structures

- Arrays, Linked Lists, Stacks, Queues
- Hash Tables (hash functions, collision handling, load factor)
- Trees (Binary, BST, B-Tree, B+ Tree, Trie)
- Graphs (adjacency list/matrix, BFS, DFS, topological sort)
- Heaps & Priority Queues

Algorithms

- Sorting (QuickSort, MergeSort, HeapSort, Radix Sort)
- Searching (Binary Search, linear scan, interpolation)
- Big-O notation (time & space complexity analysis)
- Greedy algorithms, Dynamic Programming basics

Operating Systems Concepts

- Processes vs Threads, context switching
- Memory management (virtual memory, paging, caching)
- File systems (inodes, journaling, block storage)
- Concurrency (mutex, semaphore, deadlock, race conditions)
- I/O models (blocking, non-blocking, async, multiplexing)

1.2 Linux & Command Line Mastery

Linux Fundamentals

- File system hierarchy (/, /etc, /var, /home, /opt, /tmp)
- File permissions (chmod, chown, chgrp, umask, ACLs)
- Package management (apt, yum/dnf, snap)
- Process management (ps, top, htop, kill, nice, nohup)
- Systemd services (systemctl, journalctl, unit files)

Shell Scripting (Bash)

- Variables, arrays, conditionals (if/elif/else, case)
- Loops (for, while, until), functions, exit codes
- Text processing: grep, sed, awk, cut, sort, uniq, tr, wc
- Piping (|), redirection (>, >>, 2>&1, tee)
- Regular expressions (basic & extended)
- Cron jobs & scheduling (crontab -e, cron syntax)

Essential CLI Tools

- curl, wget, jq (JSON processing)
- xargs, find, locate
- ssh, scp, rsync
- tar, gzip, zstd, zip/unzip

1.3 Networking Fundamentals

- OSI model & TCP/IP stack
- HTTP/HTTPS (methods, status codes, headers, cookies, TLS/SSL)
- REST APIs (verbs, resources, status codes, pagination)
- DNS resolution, CDNs, load balancers (L4 vs L7)
- Firewalls (iptables/nftables, security groups)
- VPN, SSH tunneling, port forwarding

1.4 Version Control – Git

- Core concepts: commits, staging, branches, tags
- Branching strategies (GitFlow, trunk-based, GitHub Flow)
- Merging vs Rebasing (when to use each)
- Pull Requests / Merge Requests, code review best practices
- CI/CD basics (GitHub Actions, GitLab CI overview)
- Pre-commit hooks, .gitignore, Git LFS

1.5 Python Deep Dive

Core Python

- Data types, collections (list, dict, set, tuple, deque)
- Control flow, functions, comprehensions, lambda
- File I/O (text, CSV, JSON, binary)
- Exception handling (try/except/finally, custom exceptions)

Intermediate Python

- OOP (classes, inheritance, polymorphism, abstract classes)
- Decorators, generators, iterators, context managers
- Type hints (typing module, mypy)
- Modules, packages, virtual environments (venv, poetry)
- Logging (logging module, handlers, formatters)

Data Engineering Python

- Pandas (DataFrame, Series, groupby, merge, pivot, window functions)
- NumPy (arrays, vectorization, broadcasting)
- Requests & httpx (REST API consumption)
- SQLAlchemy (ORM, Core, engine, sessions, migrations)
- argparse / click (CLI tools)
- Multiprocessing, threading, asyncio (concurrent data processing)

- pytest (fixtures, parametrize, mocking, conftest)

1.6 Scala Fundamentals

- val/var, type inference, functions, case classes
- Collections API (List, Map, Set, Seq, Option, Either)
- Pattern matching, for-comprehensions
- Traits, implicits, type classes
- SBT build tool (build.sbt, dependencies, tasks)
- Functional programming patterns for data processing

1.7 Java Basics (for JVM Ecosystem)

- JVM architecture (ClassLoader, heap, stack, GC)
- Core syntax, OOP, generics, collections framework
- Maven / Gradle (dependency management, build lifecycle)
- JVM tuning (-Xmx, -Xms, GC algorithms, JMX monitoring)

PHASE 2

Database Systems & SQL Mastery *(6–8 weeks)*

2.1 RDBMS Concepts & Theory

- Database architecture (storage engine, query engine, buffer pool)
- ACID properties (Atomicity, Consistency, Isolation, Durability)
- CAP theorem (Consistency, Availability, Partition tolerance)
- OLTP vs OLAP (transactional vs analytical workloads)
- Row-store vs Column-store (trade-offs, use cases)
- Normalization (1NF, 2NF, 3NF, BCNF) & denormalization
- ER modeling (entities, relationships, cardinality)

2.2 Advanced SQL

Core SQL

- DDL (CREATE, ALTER, DROP, TRUNCATE) & DML (INSERT, UPDATE, DELETE, MERGE)
- JOINs (INNER, LEFT, RIGHT, FULL, CROSS, SELF, LATERAL)
- Subqueries (scalar, correlated, EXISTS, IN)
- Set operations (UNION, INTERSECT, EXCEPT / MINUS)

Window Functions

- ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE()
- Aggregate windows (SUM, AVG, COUNT with PARTITION BY / ORDER BY)
- LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE(), NTH_VALUE()
- Frame specification (ROWS BETWEEN, RANGE BETWEEN)

Advanced Constructs

- CTEs (Common Table Expressions) & Recursive CTEs
- Pivot / Unpivot (CROSS APPLY, LATERAL, conditional aggregation)
- JSON functions (JSON_EXTRACT, JSON_ARRAY_ELEMENTS, ->, ->>)
- Regular expressions in SQL (REGEXP, SIMILAR TO)
- Date/time functions & temporal queries
- CASE / COALESCE / NULLIF / GREATEST / LEAST

2.3 Query Performance & Optimization

- EXPLAIN / ANALYZE (reading execution plans, cost estimation)
- Index types: B-Tree, Hash, GiST, GIN, BRIN, partial, composite, covering
- Query optimization (predicate pushdown, join reordering, statistics)
- Table partitioning (Range, List, Hash partitioning)
- Materialized views (refresh strategies, query rewriting)
- Stored procedures, triggers, user-defined functions

- VACUUM / ANALYZE (dead tuple cleanup, statistics updates)

2.4 PostgreSQL (Primary RDBMS)

- Installation, configuration (postgresql.conf, pg_hba.conf)
- psql CLI, pgAdmin, DBeaver
- Advanced data types: JSONB, ARRAY, UUID, HSTORE, composite, range
- Extensions (pg_stat_statements, PostGIS, pg_trgm, pg_cron)
- Full-text search (tsvector, tsquery, GIN indexes)
- Replication (streaming, logical replication, pg_basebackup)
- Backup/restore (pg_dump, pg_restore, WAL archiving, PITR)
- Connection pooling (PgBouncer, pgpool-II)

2.5 MySQL

- InnoDB vs MyISAM (ACID, locking, full-text, foreign keys)
- MySQL syntax differences from PostgreSQL
- Binary log (row-based, statement-based, mixed replication)
- Performance tuning (slow query log, query cache, buffer pool)

PHASE 3

Hadoop Ecosystem & Big Data *(8–10 weeks)*

3.1 Hadoop Architecture & History

- Origin: Google GFS, MapReduce, Bigtable papers
- Distributions: Cloudera (CDP), Hortonworks, AWS EMR, GCP Dataproc
- Cluster architecture, daemons, configuration files
- Hadoop 2.x vs 3.x (YARN, erasure coding, timeline service)

3.2 HDFS (Hadoop Distributed File System)

Architecture

- NameNode, DataNode, Secondary NameNode (checkpoint)
- Block storage (128MB default, replication factor 3)
- Write path (pipeline replication) & Read path
- Federation (multiple NameNodes) & High Availability (Active/Standby + ZooKeeper)
- Rack awareness, data locality

Operations

- CLI commands: hdfs dfs -ls, -put, -get, -cat, -mkdir, -rm, -chmod, -chown
- WebHDFS REST API, HttpFS
- Snapshots, quotas, safe mode

File Formats & Compression

- **Text/CSV:** simple but no schema, no compression optimization
- **SequenceFile:** binary key-value pairs, splittable
- **Avro:** row-based, schema evolution, compact binary, Kafka-optimized
- **Parquet:** columnar, excellent compression, predicate pushdown, analytics-optimized
- **ORC:** columnar, Hive ecosystem, ACID support, lightweight indexing
- **Compression codecs:** Snappy (fast), Gzip (high ratio), LZO (splittable), Zstandard (best balance)
- Splittable vs non-splittable compression (critical for MapReduce)
- Erasure coding (space-efficient alternative to replication)

3.3 MapReduce

- Paradigm: Map → Shuffle & Sort → Reduce
- InputFormat, InputSplit, RecordReader
- Mapper, Combiner (local reducer), Partitioner
- Reducer, OutputFormat
- Counters, Distributed Cache
- Java & Python (Hadoop Streaming) implementations
- Design patterns: filtering, summarization, joins (map-side, reduce-side, replicated)

- Chaining jobs, job sequencing

3.4 YARN (Yet Another Resource Negotiator)

- ResourceManager (scheduler + application manager)
- NodeManager, ApplicationMaster, Containers
- Schedulers: FIFO, Capacity Scheduler, Fair Scheduler
- Queue management, resource allocation, preemption
- Application lifecycle, CLI (yarn application), YARN UI

3.5 Hive

- Architecture: Metastore, Driver, Compiler, Optimizer, Executor
- HiveQL (SQL-like syntax for batch processing)
- Managed vs External tables
- Partitioning (static, dynamic, multi-level) & Bucketing
- File formats & SerDe (JSON, CSV, RegEx, Avro, ORC, Parquet)
- User-Defined Functions (UDF, UDAF, UDTF)
- Execution engines: Hive on Tez, Hive on Spark
- LLAP (Live Long and Process) – interactive queries
- Hive Metastore as universal metadata catalog

3.6 Pig

- Pig Latin: LOAD, STORE, FILTER, GROUP, FOREACH, JOIN, ORDER, DISTINCT
- Execution modes (local, MapReduce, Tez)
- UDFs in Java/Python

3.7 HBase

- Architecture: RegionServer, HMaster, ZooKeeper coordination
- Column-family data model (row key, column families, qualifiers, timestamps)
- HBase Shell commands (create, put, get, scan, delete)
- Row key design (avoid hotspotting – salting, hashing, reverse timestamps)
- Filters (SingleColumnValueFilter, PrefixFilter, RowFilter)
- Integration: Hive-HBase, Spark-HBase, Phoenix SQL layer
- Compactions (minor vs major), region splits, pre-splitting

3.8 Sqoop

- Import (RDBMS → HDFS/Hive/HBase)
- Export (HDFS → RDBMS)
- Incremental imports (append, lastmodified)
- Connectors (MySQL, PostgreSQL, Oracle, SQL Server)
- Parallel imports (--num-mappers), direct mode
- File format options (text, Avro, Parquet, SequenceFile)

3.9 Flume

- Agent architecture: Source → Channel → Sink
- Sources: Avro, Exec, Spooling Directory, Taildir, Kafka
- Channels: Memory, File, Kafka
- Sinks: HDFS, HBase, Kafka, Elasticsearch
- Multi-hop flows, fan-in, fan-out topologies
- Interceptors (timestamp, host, regex filtering)

3.10 Oozie

- Workflow (DAG of actions – MapReduce, Pig, Hive, Shell, Spark)
- Coordinator (time-based & data-based scheduling)
- Bundle (collection of coordinators)
- workflow.xml structure, parameterization (EL expressions)

3.11 Hadoop Security

- **Kerberos:** KDC, principals, keytabs, kinit, ticket-based authentication
- **Apache Ranger:** fine-grained access control, policies, auditing
- **Apache Knox:** gateway / perimeter security, REST API proxy
- **HDFS Encryption:** Transparent Data Encryption (TDE), KMS
- **Wire Encryption:** SSL/TLS for RPC and HTTP

3.12 ZooKeeper

- Coordination service for distributed systems
- Znodes (ephemeral, persistent, sequential)
- Leader election, configuration management, distributed locking
- CLI: create, get, set, ls, stat, delete

PHASE 4

Data Processing & Compute Engines *(8–10 weeks)*

4.1 Apache Spark – Core Architecture

- Driver Program, Executors, Cluster Manager
- Cluster modes: Standalone, YARN, Mesos, Kubernetes
- Deploy modes: Client vs Cluster (when to use each)

RDD (Resilient Distributed Datasets)

- Immutability, partitioning, lineage (DAG)
- Transformations: map, flatMap, filter, reduceByKey, groupByKey, join, union
- Actions: collect, count, take, saveAsTextFile, foreach
- Narrow vs Wide transformations (shuffle boundary)

Execution Model

- DAG: Jobs → Stages → Tasks
- Spark UI (stages, tasks, storage, SQL tab, event timeline)
- Persistence & caching (MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, OFF_HEAP)
- Broadcast variables & Accumulators
- Serialization: Java vs Kryo (performance implications)

4.2 Spark Performance Tuning

- **Data Skew:** salting, repartitioning, broadcast joins for skewed keys
- **Shuffle Optimization:** spark.sql.shuffle.partitions, coalesce vs repartition
- **Memory Management:** executor/driver memory, off-heap, unified memory model
- **Executor Tuning:** cores per executor, number of executors, dynamic allocation
- **Broadcast Joins:** broadcast threshold, explicit broadcast hint
- **AQE:** Adaptive Query Execution (coalesce shuffle partitions, skew join optimization)
- Spill to disk analysis, speculation mode

4.3 PySpark

DataFrame API

- SparkSession creation and configuration
- Operations: select, filter/where, withColumn, agg, groupBy, join, orderBy
- Column operations: col(), lit(), when(), cast(), alias(), isNull()
- SQL mode: registerTempView, spark.sql()

UDFs & Pandas Integration

- @udf decorator, pandas_udf (Scalar, Grouped Map, Grouped Agg)
- toPandas(), createDataFrame from Pandas, Apache Arrow optimization

Data I/O

- Reading/Writing: CSV, JSON, Parquet, ORC, Avro, JDBC, Delta, Hudi, Iceberg
- Schema enforcement: StructType, StructField, data types
- Complex types: ArrayType, MapType, StructType, explode, posexplode, flatten

4.4 Spark SQL & Catalyst

- Catalyst optimizer: Logical Plan → Optimized Plan → Physical Plan
- Cost-Based Optimization (CBO), statistics collection
- Tungsten engine: code generation, memory management
- Window functions (identical to SQL – OVER, PARTITION BY, ORDER BY)
- Hive Metastore integration (external catalog)
- Data sources API V2 (custom connectors)
- Temporary vs Global Temporary Views

4.5 Spark Structured Streaming

- Micro-batch processing vs Continuous processing (experimental)
- **Input Sources:** Kafka, File (CSV/JSON/Parquet), Socket, Rate
- **Output Sinks:** Kafka, File, Console, Memory, Foreach/ForeachBatch
- **Output Modes:** Append, Complete, Update
- Watermarking (late data handling, event-time processing)
- Stateful operations: windowed aggregations, mapGroupsWithState, flatMapGroupsWithState
- Checkpointing (exactly-once semantics, recovery)
- Stream-stream joins & stream-static joins

4.6 Spark MLlib (Overview)

- MLlib vs ML package (RDD-based vs DataFrame-based)
- Pipeline API: Transformer, Estimator, Pipeline, PipelineModel
- Feature engineering: VectorAssembler, StringIndexer, OneHotEncoder, StandardScaler
- Algorithms: Linear/Logistic Regression, Random Forest, GBT, K-Means
- Model evaluation, hyperparameter tuning (CrossValidator, ParamGridBuilder)

4.7 Apache Flink

- Architecture: JobManager, TaskManager, slots, parallelism
- DataStream API (event-driven processing)
- Time semantics: event time, processing time, ingestion time
- Windowing: Tumbling, Sliding, Session, Global windows
- State management: keyed state, operator state, state backends (RocksDB, HashMap)
- Checkpointing & Savepoints (exactly-once semantics)
- Flink SQL / Table API (stream & batch SQL)
- CDC connectors, temporal joins
- **Flink vs Spark Streaming:** true streaming vs micro-batch, latency, state management

4.8 Apache Beam

- Unified model: Pipeline, PCollection, PTransform
- Runners: DirectRunner, DataflowRunner, FlinkRunner, SparkRunner
- ParDo / DoFn (parallel processing)
- GroupByKey, CoGroupByKey, Combine
- Windowing: Fixed, Sliding, Session, Global
- Triggers & accumulation modes
- Side inputs / Side outputs

4.9 Presto / Trino

- Architecture: Coordinator, Workers, Connectors
- SQL federation across heterogeneous sources (Hive, Cassandra, MySQL, Kafka, S3)
- Memory management, query scheduling
- **Presto vs Trino:** fork history, community, enterprise offerings

PHASE 5

Data Integration, Streaming & CDC *(6–8 weeks)*

5.1 Apache Kafka – Core

Architecture

- Brokers, Topics, Partitions, Offsets
- Producers, Consumers, Consumer Groups
- ZooKeeper (legacy) vs KRaft mode (metadata quorum)

Producers

- Partitioning strategy (key-based, round-robin, custom)
- Acknowledgments: acks=0, acks=1, acks=all
- Idempotent producer (exactly-once per partition)
- Batching (batch.size, linger.ms), compression (snappy, lz4, zstd)

Consumers

- Consumer groups, partition assignment (Range, RoundRobin, Sticky, Cooperative)
- Offset management: auto-commit vs manual commit
- Consumer rebalancing protocols

Delivery & Storage

- Delivery semantics: at-most-once, at-least-once, exactly-once (EOS)
- Log segments, retention (time-based, size-based), compaction
- ISR (In-Sync Replicas), leader election, min.insync.replicas

5.2 Kafka Ecosystem

- **Kafka Connect:** source & sink connectors, distributed mode, Single Message Transforms (SMT)
- Key connectors: JDBC, S3, Elasticsearch, MongoDB, BigQuery, Debezium
- **Schema Registry:** Avro, Protobuf, JSON Schema; compatibility modes (BACKWARD/FORWARD/FULL)
- **Kafka Streams:** KTable, KStream, joins, windowing, interactive queries
- **ksqlDB:** streaming SQL, materialized views, push/pull queries
- REST Proxy, MirrorMaker 2 (cross-cluster replication)

Kafka Operations

- CLI: kafka-topics.sh, kafka-console-producer/consumer.sh, kafka-consumer-groups.sh
- Lag monitoring, offset reset strategies
- Monitoring: JMX metrics, Kafka Exporter, Prometheus + Grafana dashboards
- Performance tuning: batch.size, linger.ms, compression.type, num.partitions

5.3 Debezium – Change Data Capture (CDC)

- Log-based CDC vs query-based CDC (advantages of log-based)
- Architecture: Kafka Connect-based, reads database transaction logs
- **Supported databases:** MySQL (binlog), PostgreSQL (WAL/pgoutput), MongoDB (oplog), Oracle (LogMiner), SQL Server
- Event structure: before/after state, operation type (c/r/u/d)
- Snapshot modes: initial, schema_only, never, when_needed
- Transforms: SMTs, ExtractNewRecordState, outbox pattern
- Schema evolution (DDL changes, schema history topic)
- Debezium Server (standalone mode without Kafka Connect)

5.4 Apache NiFi

- Concepts: FlowFile, Processor, Connection, Process Group, Controller Service
- Drag-and-drop UI, templates, versioning (NiFi Registry)
- Key processors: GetFile, PutFile, ConvertRecord, ExecuteSQL, PutKafka, PutHDFS, InvokeHTTP
- Data provenance (full lineage tracking)
- Back pressure (queue tuning, flow control)
- Clustering (zero-leader architecture)
- MiNiFi (lightweight edge agent)

5.5 Cloud Streaming Services

AWS Kinesis

- Data Streams (shards, partition keys, retention 24h–365 days)
- Data Firehose (near-real-time delivery to S3/Redshift/Elasticsearch)
- Data Analytics (SQL and Flink on streaming data)
- KCL (Kinesis Client Library), checkpointing

GCP Pub/Sub

- Topics, Subscriptions (Pull/Push), global distribution
- Message ordering (ordering keys), dead-letter topics
- Pub/Sub Lite (zonal, lower-cost)
- Dataflow integration

Apache Pulsar

- Brokers, BookKeeper, ZooKeeper; multi-tenancy
- Tiered storage (S3/GCS/Azure Blob)
- Pulsar Functions, Pulsar IO (connectors)
- **Pulsar vs Kafka:** segmented architecture, geo-replication, multi-tenancy

5.6 Event-Driven Architecture Patterns

- Event sourcing (append-only event log as source of truth)
- CQRS (Command Query Responsibility Segregation)
- Saga pattern (choreography vs orchestration)

- Schema evolution strategies (backward/forward compatible)
- Idempotency patterns, transactional outbox

PHASE 6

Workflow Orchestration & Scheduling *(4–6 weeks)*

6.1 Cron & Systemd Timers

- Cron syntax (minute, hour, day, month, weekday)
- crontab management (crontab -e/-l, user vs system)
- Best practices: logging, error notifications, lock files (flock)
- Systemd timers (.timer/.service units, calendar events)

6.2 Apache Airflow (Deep Dive)

Architecture

- Components: Webserver, Scheduler, Executor, Metadata DB, Workers
- Executors: SequentialExecutor, LocalExecutor, CeleryExecutor, KubernetesExecutor

DAG Development

- DAG definition: dag_id, schedule_interval, start_date, catchup, tags
- Operators: BashOperator, PythonOperator, PostgresOperator, SparkSubmitOperator
- Cloud operators: S3ToGCSOperator, BigQueryInsertJobOperator, DataprocSubmitJobOperator
- Sensors: FileSensor, S3KeySensor, ExternalTaskSensor, HttpSensor, SqlSensor

TaskFlow API

- @task decorator, automatic XCom passing
- Dynamic task mapping (expand, partial)
- XCom: cross-task communication (push/pull, size limits)

Advanced Features

- Connections / Hooks (database & cloud connections, custom hooks)
- Variables & Pools (global config, concurrency control)
- Branching (BranchPythonOperator, conditional execution)
- Trigger rules: all_success, all_failed, one_success, none_failed, all_done
- SubDAGs vs TaskGroups (prefer TaskGroups)
- Dynamic DAG generation (factory functions, config-driven)

Operations

- CLI: trigger, backfill, test, list_dags, clear, pause/unpause
- UI: Tree view, Graph view, Gantt chart, task logs
- Deployment: Docker Compose, Helm on K8s, MWAA (AWS), Cloud Composer (GCP)
- Monitoring: StatsD, Prometheus, SLA alerting
- Best practices: idempotent tasks, no side effects in DAG files, atomicity, testing
- Security: RBAC, LDAP/OAuth integration, Fernet key encryption

6.3 Luigi

- Concepts: Task, Target, Parameter, Worker, Central Scheduler
- Task dependencies (requires(), output())
- Targets: LocalTarget, S3Target, HDFSTarget
- Parameters: IntParameter, DateParameter, ListParameter
- Luigi daemon (central scheduler UI, task visualization)
- **Luigi vs Airflow:** simple dependency resolution vs full orchestration

6.4 Prefect

- Concepts: Flows, Tasks, Flow Runs, Deployments, Work Pools
- Prefect 2.x/3.x (@flow/@task decorators, Pythonic API)
- State handling: Pending, Running, Completed, Failed, Retrying, Cancelled
- Retries, caching, scheduling (Cron, Interval, RRule)
- Prefect UI (flow run tracking, deployment management)
- Prefect Cloud vs Server (managed vs self-hosted)
- Infrastructure blocks (Docker, K8s, Process, ECS)
- **Prefect vs Airflow:** Pythonic, no DAG constraints, dynamic pipelines

6.5 Dagster

- Concepts: Assets, Ops, Jobs, Graphs, Resources, IO Managers
- Software-Defined Assets (declarative pipeline definition)
- Dagster UI / Dagit (asset lineage, run monitoring, launchpad)
- Partitions & Backfills (time-based, incremental processing)
- Schedules & Sensors (time-based, event-driven triggers)
- Resources (dependency injection), IO Managers (S3, BigQuery, Snowflake)
- Testing: unit testing ops/assets, in-memory IO managers
- **Dagster vs Airflow:** asset-centric vs task-centric, type system, testability

6.6 Mage.ai

- Concepts: Blocks (Loader, Transformer, Exporter), Pipelines, Triggers
- Interactive notebook-style block development
- Pipeline types: standard, streaming, integration
- Templates, reusability, built-in testing
- **Mage vs Airflow:** notebook-first, faster iteration, built-in testing

PHASE 7

NoSQL, Data Warehousing & Modeling *(8–10 weeks)*

7.1 MongoDB

- Data model: collections, documents, BSON format
- Architecture: mongod, mongos, config servers
- CRUD: insertOne, find, updateOne, deleteOne, bulkWrite
- Aggregation framework: $match, $group, $project, $unwind, $lookup, $sort, $limit, $facet
- Indexing: single field, compound, multikey, text, geospatial, TTL, wildcard
- Replication: Replica Sets, primary/secondary, read preferences
- Sharding: shard key selection (ranged vs hashed), balancer, chunks
- Schema design patterns: embedding vs referencing, denormalization, bucket pattern
- Change Streams (real-time data change notifications)
- MongoDB Atlas (managed cloud, Atlas Search, Data Federation)

7.2 Apache Cassandra

- Architecture: peer-to-peer, gossip protocol, consistent hashing, vnodes
- Data model: keyspace, table, partition key, clustering key, static columns
- CQL: SELECT, INSERT, UPDATE, DELETE, TTL, BATCH
- Write path: commit log → memtable → SSTable flush → compaction
- Read path: memtable + SSTables, bloom filters, key/row cache
- Consistency levels: ONE, QUORUM, ALL, LOCAL_QUORUM
- Data modeling: query-first design, denormalization, materialized views
- Anti-patterns: secondary indexes on high-cardinality, large partitions

7.3 Redis

- Data structures: String, List, Set, Sorted Set, Hash, Stream, HyperLogLog, Bitmap
- Persistence: RDB snapshots, AOF, hybrid
- Pub/Sub, Redis Streams (consumer groups)
- Redis Cluster (sharding, replication, automatic failover)
- Caching patterns: cache-aside, write-through, write-behind, TTL strategies
- Use cases: caching, session storage, rate limiting, real-time leaderboards

7.4 Neo4j

- Graph model: nodes, relationships, properties, labels
- Cypher query language: MATCH, CREATE, MERGE, WHERE, RETURN
- Graph algorithms: shortest path, PageRank, community detection
- Use cases: knowledge graphs, data lineage, recommendation engines, fraud detection

7.5 Elasticsearch

- Architecture: index, shard, replica, cluster, node
- Mappings, data types (keyword, text, nested, geo_point)
- Querying: match, term, bool, range, aggregations, full-text search
- ELK Stack (Elasticsearch + Logstash + Kibana)
- Use cases: log analysis, search, metrics aggregation

7.6 Data Warehousing Concepts

Architecture Paradigms

- Data Warehouse vs Data Lake vs Data Lakehouse
- ETL vs ELT (when to use each)
- Batch vs Real-time vs Lambda vs Kappa architecture

Dimensional Modeling

- Star Schema (fact tables, dimension tables, measures, foreign keys)
- Snowflake Schema (normalized dimensions)
- Galaxy Schema (multiple fact tables sharing dimensions)

Slowly Changing Dimensions (SCD)

- Type 0 (fixed), Type 1 (overwrite), Type 2 (history tracking)
- Type 3 (previous value column), Type 4 (mini-dimension), Type 6 (hybrid)

Dimension & Fact Types

- Surrogate keys vs natural keys, junk dimensions, degenerate dimensions
- Role-playing dimensions, conformed dimensions
- Fact types: Transaction, Periodic Snapshot, Accumulating Snapshot, Factless
- Data marts (subject-area-specific subsets)

7.7 Data Modeling Methodologies

- **Kimball:** bottom-up, dimensional modeling, Bus Architecture, Conformed Dimensions
- **Inmon:** top-down, enterprise data warehouse, 3NF normalized
- **Data Vault 2.0:** Hubs (business keys), Links (relationships), Satellites (descriptive data)
- Data Vault advantages: auditability, agility, historical tracking, parallel loading
- **One Big Table (OBT):** denormalized single table for analytics (trade-offs)
- **Activity Schema:** event-based modeling for analytics
- **Anchor Modeling:** highly normalized, temporal data modeling

7.8 Google BigQuery

- Architecture: Dremel (compute) + Colossus (storage) + Borg + Jupiter (network)
- Standard SQL with nested/repeated fields (UNNEST, STRUCT, ARRAY)
- Partitioning: ingestion time, column-based (date/timestamp/integer)
- Clustering (multi-column, automatic re-clustering)
- BigQuery ML (CREATE MODEL for ML directly in SQL)

- BigQuery Storage API (fast bulk reads for Spark/Pandas)
- Materialized views, scheduled queries, BI Engine caching
- Cost optimization: on-demand vs flat-rate, slot reservations
- BigQuery Data Transfer Service

7.9 Snowflake

- Architecture: Storage / Compute (Virtual Warehouses) / Cloud Services
- Virtual warehouses: auto-scaling, auto-suspend, size selection
- Time Travel (query historical data up to 90 days), UNDROP
- Zero-copy cloning (instant cloning for dev/test)
- Snowpipe (continuous loading from S3/GCS/Azure Blob)
- Streams & Tasks (CDC on Snowflake tables, task scheduling)
- Secure Data Sharing (exchange without copying)
- Stages (internal, external), file formats
- Dynamic Tables (materialized incremental transformations)

7.10 Amazon Redshift

- Architecture: Leader node, Compute nodes, slices
- Distribution styles: EVEN, KEY, ALL, AUTO
- Sort keys: compound vs interleaved
- Redshift Spectrum (query S3 data directly)
- Redshift Serverless (on-demand, pay-per-query)
- Concurrency Scaling, COPY command (bulk loading from S3)

7.11 ClickHouse & Druid

ClickHouse

- Column-oriented storage (MergeTree engine family)
- Real-time analytics (sub-second query performance)
- MaterializedView, AggregatingMergeTree (pre-aggregation)
- Distributed tables (cross-shard queries)

Apache Druid

- Real-time OLAP (segment-based architecture, roll-up)
- Ingestion: batch (Hadoop/S3), streaming (Kafka)
- Use cases: time-series analytics, dashboards, ad-tech, monitoring

7.12 Open Table Formats (Data Lakehouse)

- **Delta Lake:** ACID transactions, time travel, schema evolution, Z-ordering
- **Apache Iceberg:** hidden partitioning, partition evolution, snapshot isolation
- **Apache Hudi:** Copy-on-Write vs Merge-on-Read, incremental processing, CDC support
- Comparison: Delta vs Iceberg vs Hudi (ecosystem, community, features)
- Enabling Data Lakehouse architecture on cloud storage

PHASE 8

Cloud Platforms, DataOps, Governance & MLOps *(8–12 weeks)*

8.1 Google Cloud Platform (GCP)

- **Cloud Storage (GCS):** buckets, object lifecycle, storage classes, gsutil CLI
- **BigQuery:** covered in Phase 7
- **Cloud Dataproc:** managed Spark/Hadoop, autoscaling, initialization actions
- **Cloud Dataflow:** managed Apache Beam, streaming/batch, auto-scaling, templates
- **Cloud Pub/Sub:** covered in Phase 5
- **Dataplex:** data mesh management, data zones, discovery, quality rules
- **Cloud Composer:** managed Airflow on GKE
- **Cloud Data Fusion:** visual ETL/ELT, code-free pipeline builder (CDAP-based)
- **Datastream:** serverless CDC from MySQL/PostgreSQL/Oracle to BigQuery/GCS
- **Cloud Functions / Cloud Run:** serverless compute for event-driven processing
- **Vertex AI:** managed ML platform, feature store, model deployment
- IAM: service accounts, roles, permissions, organization policies

8.2 Amazon Web Services (AWS)

- **S3:** buckets, storage classes, lifecycle policies, S3 Select, Event Notifications
- **AWS Glue:** serverless ETL, Crawlers, Data Catalog, Spark jobs, Glue Studio
- **EMR:** managed Spark/Hadoop/Flink/Hive, EMR on EKS, EMR Serverless
- **Kinesis:** covered in Phase 5
- **Redshift:** covered in Phase 7
- **Lambda:** serverless functions, event-driven processing, Step Functions
- **Athena:** serverless Presto/Trino on S3, pay-per-query
- **Lake Formation:** data lake setup, fine-grained access control, governed tables
- **DynamoDB:** serverless NoSQL, streams for CDC, global tables
- **MSK:** managed Apache Kafka
- **SageMaker:** ML model training, deployment, feature store

8.3 Microsoft Azure

- **ADLS Gen2:** hierarchical namespace, data lake storage (Blob + HNS)
- **Azure Data Factory:** visual ETL/ELT, copy activity, data flows, pipelines
- **Synapse Analytics:** dedicated SQL pools, Spark pools, serverless SQL
- **Azure Databricks:** managed Spark, Unity Catalog, Delta Lake integration
- **Event Hubs:** managed Kafka-compatible event streaming
- **Stream Analytics:** real-time stream processing with SQL
- **Cosmos DB:** multi-model globally-distributed NoSQL
- **Microsoft Purview:** data governance, cataloging, lineage

8.4 Infrastructure as Code (IaC)

- **Terraform:** providers (AWS/GCP/Azure), resources, state management, modules
- Terraform for data infra: BigQuery datasets, S3 buckets, EMR clusters, Kafka topics
- **Pulumi:** IaC in Python/TypeScript/Go
- CloudFormation (AWS) / Deployment Manager (GCP)
- **Ansible:** configuration management, playbooks for server setup

8.5 Docker & Kubernetes

Docker

- Dockerfile, images, containers, volumes, networks, docker-compose
- Docker for data engineering: containerized Spark, Airflow, Kafka development
- Multi-stage builds, image optimization

Kubernetes

- Pods, Deployments, Services, ConfigMaps, Secrets, Namespaces
- Kubernetes for data: SparkOnK8s, Airflow KubernetesExecutor, Strimzi (Kafka on K8s)
- Helm charts (Airflow Helm, Spark operator)
- Container registries: Docker Hub, GCR, ECR, ACR

8.6 Data Quality Tools

- **Great Expectations:** expectations, validation, data docs, checkpoints, profiling
- **Soda:** SodaCL checks, Soda Cloud, Soda Core, Airflow integration
- **AWS Deequ:** constraint verification, anomaly detection on Spark DataFrames
- **dbt Tests:** schema tests (unique, not_null, accepted_values, relationships), custom tests
- **Monte Carlo / Elementary:** data observability, anomaly detection, freshness monitoring
- Data quality dimensions: accuracy, completeness, consistency, timeliness, uniqueness, validity

8.7 Data Governance & Cataloging

Governance Pillars

- Ownership, stewardship, policies, standards, compliance

Data Cataloging

- Apache Atlas, Amundsen, DataHub, OpenMetadata
- Data lineage: tracking data flow, impact analysis

Dataplex (GCP)

- Data mesh-style governance, data quality rules, data zones

Privacy & Compliance

- PII handling: masking, tokenization, anonymization, pseudonymization
- Regulations: GDPR, CCPA, HIPAA compliance requirements
- Data contracts: schema agreements between producers & consumers

- Access control: RBAC, ABAC, column-level, row-level security

8.8 dbt (Data Build Tool)

Core Concepts

- Models (SQL SELECT), refs, sources, seeds, snapshots
- Project structure: models/, macros/, seeds/, tests/, snapshots/
- Materializations: view, table, incremental, ephemeral

Advanced

- Jinja templating: macros, control structures, ref(), source()
- Testing: schema tests, data tests, custom generic tests
- Documentation: doc blocks, generated documentation site
- Packages: dbt-utils, dbt-expectations, packages.yml
- dbt Cloud vs dbt Core (managed IDE vs CLI)
- Orchestration integration: dbt with Airflow / Dagster / Prefect

8.9 CI/CD for Data Pipelines

- Version control: DAGs, SQL, schemas, configs in Git
- Testing strategies: unit tests (dbt/pytest), integration tests, data validation
- CI/CD tools: GitHub Actions, GitLab CI, Jenkins, CircleCI
- Environment management: dev, staging, production data environments
- Blue-green deployments for data pipelines
- Schema migration tools: Flyway, Alembic, Liquibase
- Feature flags for gradual rollout of pipeline changes
- Monitoring & alerting: Prometheus, Grafana, PagerDuty, Datadog

8.10 Data Architecture Patterns

- **Data Mesh:** domain-oriented ownership, data as a product, self-serve platform, federated governance
- **Data Lakehouse:** combining data lake + warehouse, open table formats
- **Medallion Architecture:** Bronze (raw) → Silver (cleansed) → Gold (aggregated)
- **Lambda Architecture:** batch + speed layers, serving layer
- **Kappa Architecture:** streaming-only (simplification of Lambda)
- **Event-Driven Architecture:** events as first-class citizens
- **Reverse ETL:** syncing warehouse data back to operational systems (Census, Hightouch)

8.11 MLOps Fundamentals (for Data Engineers)

- ML pipeline stages: data collection, feature engineering, training, validation, deployment, monitoring
- **Feature Stores:** Feast, Tecton, Vertex AI Feature Store, Hopsworks
- **Experiment Tracking:** MLflow, Weights & Biases, Neptune.ai
- **Model Registries:** MLflow Model Registry, SageMaker Model Registry

- **Model Serving:** TensorFlow Serving, Seldon Core, BentoML, KServe
- **Data Versioning:** DVC (Data Version Control), LakeFS
- Data engineers build the infrastructure and pipelines that support ML workflows

8.12 Capstone Projects

- **Project 1:** End-to-end batch pipeline (CSV/API → Spark → BigQuery, orchestrated by Airflow, Great Expectations quality checks)
- **Project 2:** Real-time streaming (Kafka + Debezium CDC from PostgreSQL → Spark Structured Streaming → Delta Lake → Grafana dashboard)
- **Project 3:** Data Lakehouse (S3/GCS raw zone, Bronze-Silver-Gold Medallion, dbt transforms, Iceberg table format, Trino federated queries)
- **Project 4:** Cloud-native GCP pipeline (Pub/Sub + Dataflow + BigQuery + Dataplex governance + Cloud Composer orchestration)
- **Project 5:** Data Mesh prototype (domain-owned datasets, data contracts, self-serve platform, DataHub catalog)

Additional Resources & Learning Tips

File Formats Quick Reference

- **CSV/TSV:** simple text, universal support, no schema, no compression
- **JSON/JSONL:** semi-structured, human-readable, verbose, line-delimited for streaming
- **Avro:** row-based binary, schema evolution, compact, Kafka-optimized
- **Parquet:** columnar binary, excellent compression, predicate pushdown, analytics-optimized
- **ORC:** columnar binary, Hive ecosystem, ACID support, lightweight indexing
- **Protocol Buffers:** Google's binary serialization, language-neutral, Schema Registry support

Compression Codecs

- **Snappy:** fast compress/decompress, moderate ratio, Spark default
- **Gzip:** high compression ratio, slower, NOT splittable
- **LZO:** splittable (with index), fast, Hadoop-optimized
- **Zstandard (zstd):** excellent ratio + speed, modern choice, tunable compression level
- **Brotli:** very high compression, slower, web-optimized

Learning Path Tips

- Follow the phase sequence – each phase builds on the previous one
- Build hands-on projects at every phase (theory alone is insufficient)
- Practice SQL daily – it is the most-used skill in data engineering
- Read original papers: Google's GFS, MapReduce, Bigtable, Dremel papers are foundational
- Join communities: r/dataengineering, dbt Slack, Apache mailing lists, Data Engineering Weekly
- Use cloud free tiers: GCP $300 credit, AWS Free Tier, Azure $200 credit
- Build a portfolio: GitHub repos with well-documented data projects
- Contribute to open-source data tools for learning and networking

Key Job Requirement Technologies (Quick Map)

- **PySpark:** Phase 4 (4.3)
- **GCP Stack:** Phase 8 (8.1) + BigQuery in Phase 7
- **Advanced SQL:** Phase 2 (2.2–2.3)
- **NoSQL / MongoDB:** Phase 7 (7.1)
- **Scala & Python:** Phase 1 (1.5–1.6)
- **Data Quality Tools:** Phase 8 (8.6)
- **Airflow / Prefect / Luigi:** Phase 6 (6.2–6.4)
- **Data Governance:** Phase 8 (8.7)
- **DWH & Data Modeling:** Phase 7 (7.6–7.7)